COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	10/20/2026	**DATE SUBMITTED:**	10/20/2026
NAME:	Ralf Nicolo B. Abello Rodney C. Domingo	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14

1. PROCEDURE OUTPUT

	Best Case	Worst Case	Analysis
Algorithm 1: Bubble Sort	O(n)	O(n^2)	I found that bubble sort is O(n) in the best case, when the data is already sorted, and O(n^2) in the worst case, when the data is in reverse order. This makes it very slow for large n, but it needs only O(1) space.
Algorithm 2: Merge Sort	O(n log n)	O(n log)	I found that merge sort is O(n log n) in both the best and worst case, because it always splits the data in half and merges it back. It is much faster than bubble sort on large inputs, but it needs O(n) extra space for merging.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Algorithm 1	Algorithm 2

Best Case	**N = 100**
	`Best Case 0.0003 ms`
	N = 1,000
	`Best Case 0.0009 ms`
	N = 10,000
		N = 100
		`Input Size N = 100`
		`- Best Case : 0.0197 ms`
		N = 1,000

	Best Case 0.0077 ms	Input Size N = 1000 - Best Case : 0.1726 ms **N = 10,000** Input Size N = 10000 - Best Case : 1.8052 ms
Worst Case	**N = 100** Worst Case 0.0114 ms **N = 1,000** Worst Case 0.9781 ms **N = 10,000** Worst Case 90.7925 ms	**N = 100** Input Size N = 100 - Worst Case : 0.0164 ms **N = 1,000** Input Size N = 1000 - Worst Case : 0.1642 ms **N = 10,000** Input Size N = 10000 - Worst Case : 1.6886 ms

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

THEORETICAL ANALYSIS

LINEAR SEARCH(A, n, key)

```
for i = 0 to n-1
  if A[i] == key
    return i
return -1
```

BINARY SEARCH(A, n, key)

```
lo = 0
hi = n-1
while lo <= hi
  mid = (lo + hi) / 2
  if A[mid] == key
    return mid
  if A[mid] < key
    lo = mid + 1
  else
    hi = mid - 1
return -1
```

In our analysis of the pseudocode, we see that linear search goes through the array one element at a time and compares each one to the key. In the best case the key is at the first position, so it takes only one comparison, O(1). In the worst case the key is last or missing, so it makes n comparisons, which gives O(n), and the average case is also O(n). Binary search needs a sorted array. It compares the key with the middle element and throws away half of the remaining range on every pass of the loop, so the range shrinks from n to n/2, n/4 and so on until it is empty. This takes at most about $\log_2 n$ comparisons, so the worst and average cases are O(log n), and the best case is O(1) when the key is the middle element. For space, both algorithms only use a few variables (i, lo, hi, mid) no matter how large n is, so both have O(1) space complexity in our iterative versions. In summary, we

found that binary search is much faster for large inputs, but it can only be used on sorted data, while linear search is slower but works on any data.

EMPERICAL ANALYSIS

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	0.0116 ms	Elapsed Time: 0.00110 ms
10,000	0.0175 ms	Elapsed Time: 0.01000 ms
100,000	0.0312 ms	Elapsed Time: 0.09520 ms

ANALYSIS

We confirmed that Bubble Sort (Algorithm 1) grows quadratically in the worst case, rising from 0.0114 ms at N = 100 to 90.7925 ms at N = 10,000. Each tenfold increase in N raised its runtime by roughly 90 times, close to the expected 100 times for O(n^2). Merge Sort (Algorithm 2) grew only about 10 times per tenfold increase, from 0.0164 ms to 1.6886 ms, which matches O(n log n). Bubble Sort was faster at N = 100, but Merge Sort was about 54 times faster at N = 10,000. This shows that the overhead of Merge Sort only pays off at larger input sizes.

For the supplemental activity, we found that Linear Search grew in proportion to N, from 0.00110 ms to 0.09520 ms, which matches its O(N) bound. Binary Search grew much more slowly, from 0.0116 ms to 0.0312 ms, which fits its O(log N) strategy. Linear Search was still faster at N = 1,000, but Binary Search was about three times faster at N = 100,000. Overall, we saw that the better algorithm depends on input size.

CONCLUSION

Abello	In this activity, I learned that theoretical Big-O analysis and experimental timing support each other. I saw that Bubble Sort slows down sharply as N grows while Merge Sort stays efficient, even though Bubble Sort can be faster on very small inputs. I also learned that Binary Search only outperforms Linear Search once the data set is large enough. These results made me realize that I should consider input size and expected data conditions when choosing an algorithm. Complexity notation alone is not enough, so I now see why running experiments matters alongside theory.
Domingo	We assessed the basic sorting and searching algorithms' theoretical and empirical performance across a range of input sizes. Experimental findings verified that lower order-of-growth algorithms, like Binary Search and Merge Sort, show noticeably better scalability than O(N^2) sorting and linear searching strategies. Therefore, in large-scale data processing applications, choosing an algorithm based on computational complexity is essential for maximizing system performance and execution efficiency.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.